Matrice de couverture des fixtures - SCE 2.0 Basic Set v1.7

Date : 2026-05-14 **Test pilote :** `crates/pos-fiscal/tests/uc_fixtures.rs` **Fixtures source :** `docs/20250916_UC_BasicSet_POS_FDM_1.7/`

Chaque dossier UC du Basic Set v1.7 publié par SPF Finances est chargé par le walker de découverte et validé. Le `*_Response.json` (ou `status_response.json`) de chaque fixture est parsé via `serde_json::from_value::<SignResult>` / `FdmStatus` - autrement dit c'est **le même désérialiseur** que le client FDM utilise en production dans `crates/pos-fiscal/src/fdm_client.rs`.

Invariants imposés par UC :

- Les **événements N** (signSale) doivent porter `shortSignature` (40 caractères hex) et `verificationUrl`.
- **Tout événement non-N** ne doit PAS porter `shortSignature` ni `verificationUrl`.
- Les **événements de formation** (label T) doivent omettre `vatCalc` (changelog v1.7 "M171 : correction dans la réponse FDM (pas de vatCalc)").
- Chaque mutation `Variables.json` porte les champs communs SCE 2.0 (`posId`, `posFiscalTicketNo`, `posDateTime`, `deviceId`, `vatNo`, `estNo`, `posSwVersion`, `bookingPeriodId`, `bookingDate`, `ticketMedium`, `employeeId`).

Matrice de couverture

Deux colonnes :

- **Parse fixture** - `crates/pos-fiscal/tests/uc_fixtures.rs` charge le `*_Response.json` publié et le désérialise via les types production `SignResult` / `FdmStatus`.
- **E2e applicatif** - `src-tauri/tests/spf_e2e_basic.rs` pilote la fonction de commande Tauri `do_*` correspondante contre un wiremock FDM amorcé avec cette fixture, puis assert sur l'archive SQLite + les compteurs + l'état de la commande.

UC	Label événement	Parse fixture	E2e applicatif
M110_signSale	N	✓	✓
M111_signSale_Refund	N	✓	✓
M112A_signSale_RefundPartial / M112A_1_OriginalSale	N	✓	- (pattern non déclaré, couvert via M112B)
M112A_signSale_RefundPartial / M112A_2_PartialRefund	N	✓	- (pattern non déclaré, couvert via M112B)
M112B_signSale_RefundPartial / M112B_1_OriginalSale	N	✓	✓ (chaîné)
M112B_signSale_RefundPartial / M112B_2_PartialRefund	N	✓	✓ (chaîné)
M121_signOrder	P	✓	✓
M122_signCostCenterChange	P	✓	- (pas d'IPC encore)
M123_signPrebill	P	✓	✓
M130_signMoneyInOut	F	✓	✓
M131_signDrawerOpen	F	✓	✓
M132_signPaymentCorrection	F	✓	- (pas d'IPC encore)
M133_signMoneyInOut_DrawerDeclaration	F	✓	- (variante de M130)
M140_signWorkIn	S	✓	✓
M141_signWorkOut	S	✓	✓
M150_signInvoice	I	✓	✓ (`sign_invoice_fiscal`, chaîné à la vente)
M160_signCopy_Sale	C	✓	✓

UC	Label événement	Parse fixture	E2e applicatif
M161_signCopy_Order	C	✓	✓ (copie d'un événement P)
M170_signWorkIn_Training	T	✓	✓
M171_signSale_Training	T	✓	✓
M171B_signSale_Training	T	✓	- (variante de M171)
M172_signCopy_Training	T	✓	✓ (copie d'un événement T)
Q100_Status	- (statut)	✓	✓ au niveau client (`client_status.rs`, voir ci-dessous)
Q110_Status_multi_lang_messages	- (statut)	✓	✓ au niveau client (`client_status.rs`)

Parse fixture : 24 / 24 passent. E2e applicatif : 16 orchestrations uniques dans `spf_e2e_basic.rs` + 4 Reports dans `spf_e2e_reports.rs`. Les lignes "variante" restantes (M133, M171B) sont couvertes transitivement par l'e2e de l'UC parent ; M112A n'est pas un pattern offert par le POS (le remboursement partiel est toujours émis en événement séparé = pattern M112B, déclaré au dossier) ; M122 et M132 sont reportés et non déclarés. M150 (événement I, e-facturation Peppol) est bien implémenté et couvert par le parse fixture et l'e2e (feature `peppol` activée par défaut pour les tests), mais ce feature flag n'est **pas** activé dans la build de certification v1.0.0 (`cert = ["custom-protocol"]`) - événement I reporté en mise à jour certifiée v1.1, voir *DECISION_PEPPOL_AP_FR.md*.

Enveloppe Q100/Q110 - divergence résolue

Les fixtures SPF Q100/Q110 enveloppent le payload sous `data.status` (minuscule), alors que le schéma POS v2.2 documente `Status` (majuscule). `FdmClient::status*` gère désormais les deux orthographes : il interroge `Status` puis retombe sur `status` si le FDM répond avec la forme minuscule. Ce contrat est verrouillé par les 3 tests wiremock de `crates/pos-fiscal/tests/client_status.rs` (argument de langue, fallback d'orthographe, parsing de l'enveloppe multi-langue Q110).

Labels d'événements exercés : les 8 sur 8 - N, P, F, S, I, C, T dans `spf_e2e_basic.rs` (I testé mais hors périmètre de la build cert v1.0.0, cf. ci-dessus), et R dans `spf_e2e_reports.rs` (M180, M182, M184) en complément du parsing `crates/pos-fiscal/tests/reports_fixtures.rs`.

Dérive de schéma v1.7 corrigée dans cette phase

Problème	Fixture qui l'a exposé	Correctif
`SignResult.footer: Vec<String>` était non-optionnel, causant l'échec de désérialisation des réponses M121/M171 parce que v1.7 omet le champ de ces réponses entièrement.	M121_signOrder, M171_signSale_Training	Ajout de `#[serde(default)]` à `footer` dans `crates/pos-fiscal/src/response.rs`. Le champ manquant désérialise désormais en `vec![]`.

Aucune dérive trouvée côté requête - chaque Variables.json dans le set contient tous les `CommonFields` et les champs spécifiques à l'événement que nos builders `*Input` émettent.

Vérifications de sanité ciblées rédigées à la main

Ce sont des `#[test]`s supplémentaires dans `uc_fixtures.rs` qui épinglent les fixtures les plus tordues sur des valeurs attendues spécifiques, afin que toute dérive future de l'API échoue bruyamment :

- `m110_sign_sale_response_shape` - chemin happy N ; compteurs 15/59 depuis la fixture ; shortSignature, verificationUrl, vatCalc tous présents.
- `m121_sign_order_response_omits_n_only_fields` - prouve que `footer` défaille à `vec![]` quand le wire l'omet.
- `m171_training_sale_has_no_vat_calc` - épinglé selon le changelog v1.7.
- `q100_status_response_shape` - épinglé pour garder le désérialiseur de statut FDM honnête.

Reproductibilité

```
cd pili-pos
cargo test --package pos-fiscal --test uc_fixtures -- --nocapture
```

Le test roll-up (`basic_set_v1_7_fixtures_deserialize_and_match_schema`) imprime la matrice complète pass/fail sur stderr à chaque exécution ; les quatre tests ciblés épinglent des UC spécifiques par nom.

Hors périmètre ici (reporté aux phases ultérieures)

- Fixtures Reports v1.3 (M180-M184) → Phase 3.
- Fixtures Advanced Set v1.5 (UC200-UC4123) → Phases 5/6/7.
- Sérialisation **requête** octet-équivalente (nous vérifions actuellement `Variables.json` pour l'ensemble de champs requis, pas pour l'égalité octet à octet). La Phase 4 introduit la sérialisation JSON canonique, après quoi nous ajouterons un test d'équivalence de requête pour le dossier cert.